

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 1, Магистратура, группа 12-25РПм
Херсонская область - 2025

Дисциплина:
«ОСНОВЫ ПРОГРАММНОЙ ИНЖЕНЕРИИ»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 1-го курса

Проверили:
АЛЕШОВ Владимир Витальевич
преподаватель кафедры, аспирант
КРУГЛЫЙ Дмитрий Георгиевич
доктор технических наук, профессор
кафедры

ПРАКТИЧЕСКАЯ РАБОТА №3
«Построение архитектуры программного средства»

I. ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

1.1. Цель работы

Изучение базовых принципов и архитектурных шаблонов построения программных приложений, выбор стратегии проектирования архитектуры, а также освоение практических навыков работы с системами контроля версий Git с использованием TortoiseGit и сервиса GitHub.

1.2. Задачи работы

- Создать архитектуру программного средства в соответствии с техническим заданием.
- Изучить архитектурные стили и принципы проектирования программных систем.
- Освоить методику работы с распределенной системой контроля версий Git.
- Приобрести навыки визуального проектирования архитектуры приложения.
- Закрепить знания по управлению версиями программного обеспечения.

II. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1. Понятие жизненного цикла программного средства

- Жизненный цикл программного средства (ЖЦ ПС) представляет собой совокупность процессов, этапов и работ, выполняемых при создании, эксплуатации и сопровождении программного обеспечения.
- Стандарты жизненного цикла позволяют унифицировать подходы к разработке и управлению программными проектами.
- В Российской Федерации базовым стандартом ЖЦ ПС является ГОСТ Р ИСО/МЭК 12207.

2.2. Процессы жизненного цикла ПО по ГОСТ Р ИСО/МЭК 12207

- Основные процессы: приобретение, поставка, разработка, эксплуатация, сопровождение.
- Вспомогательные процессы: документирование, управление конфигурацией, обеспечение качества, верификация, аттестация, аудит, разрешение проблем.
- Организационные процессы: управление, обучение, усовершенствование, создание инфраструктуры.

2.3. Архитектура программного обеспечения

- Архитектура программного обеспечения определяет структуру системы, состав компонентов и способы их взаимодействия.
- Архитектура оказывает существенное влияние на надежность, масштабируемость, производительность и сопровождаемость системы.
- Проектирование архитектуры основывается на требованиях пользователей, бизнес-целях и возможностях ИТ-инфраструктуры.

2.4. Основные принципы проектирования архитектуры

- Разделение функций и минимизация связности между компонентами.
- Принцип единственной ответственности.
- Принцип минимального знания (закон Деметера).
- Принцип «Не повторяйся» (DRY).
- Минимизация преждевременного проектирования.

2.5. Архитектурные стили и типы приложений

- Клиент-серверная архитектура.
- Многослойная и n-уровневая архитектура.
- Компонентная архитектура.
- Объектно-ориентированная архитектура.
- Сервисно-ориентированная архитектура (SOA).
- Веб-приложения, мобильные, насыщенные клиентские и сервисные приложения.

III. ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1. Построение архитектуры программного средства

- Вариант 12: архитектура информационной системы «Издательство».
- Архитектура разработана с использованием многослойного подхода.
- Выделены следующие уровни: пользовательский интерфейс, бизнес-логика, уровень доступа к данным и база данных.
- Архитектура визуально спроектирована с использованием Microsoft Visio.
- Для моделирования использованы элементы: процесс, объект, компонент и линии связи.

3.2. Использование системы контроля версий Git

- Git является распределенной системой контроля версий.
- Для работы использована графическая надстройка TortoiseGit.
- Создан локальный репозиторий с инициализацией командой git init.
- Выполнена настройка имени пользователя и адреса электронной почты.
- Добавлены файлы в индекс и выполнен первый коммит.

3.3. Основные команды Git, использованные в работе

- git config — настройка параметров пользователя.
- git init — инициализация репозитория.
- git status — проверка состояния репозитория.
- git add — добавление файлов в область подготовки.
- git commit — фиксация изменений.

IV. ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1. Что подразумевается под созданием архитектуры приложения?

Создание архитектуры приложения — это процесс определения структуры программной системы, состава ее компонентов и способов их взаимодействия с целью обеспечения требуемых характеристик качества.

4.2. Каково основное назначение архитектуры приложения?

Основное назначение архитектуры — обеспечить упорядоченное, масштабируемое и сопровождаемое решение, соответствующее требованиям пользователей и бизнес-целям.

4.3. Основные принципы разработки архитектуры ПО

- Разделение ответственности.
- Минимальная связность компонентов.
- Единая ответственность каждого модуля.
- Повторное использование компонентов.
- Гибкость и расширяемость архитектуры.

4.4. Системы контроля версий

- Git.
- Subversion (SVN).
- Mercurial.
- Perforce.

4.5. Характеристика системы Git

Git — это распределенная система контроля версий, обеспечивающая высокую скорость работы, надежное хранение истории изменений и возможность параллельной разработки.

4.6. Что такое клонирование и синхронизация репозитория?

Клонирование — создание локальной копии удаленного репозитория.
Синхронизация — процесс обмена изменениями между локальным и удаленным репозиториями.

V. ВЫВОДЫ

5.1. Заключение

- В ходе практической работы была разработана архитектура программного средства «Издательство».
- Изучены основные архитектурные стили и принципы проектирования.
- Освоены базовые приемы работы с системой контроля версий Git.
- Получены практические навыки, необходимые для дальнейшей разработки программных проектов

Репозиторий тут:

https://github.com/Trifonix/rasp-for-mag/tree/master/1_2025-1/4_osn-pro-inj/publishing-architecture